KU LEUVEN

Geavanceerde Computerarchitectuur 2nd session

Race conditions

Thomas Feys, thomas.feys@kueluven.be
DRAMCO – WaveCore – ESAT

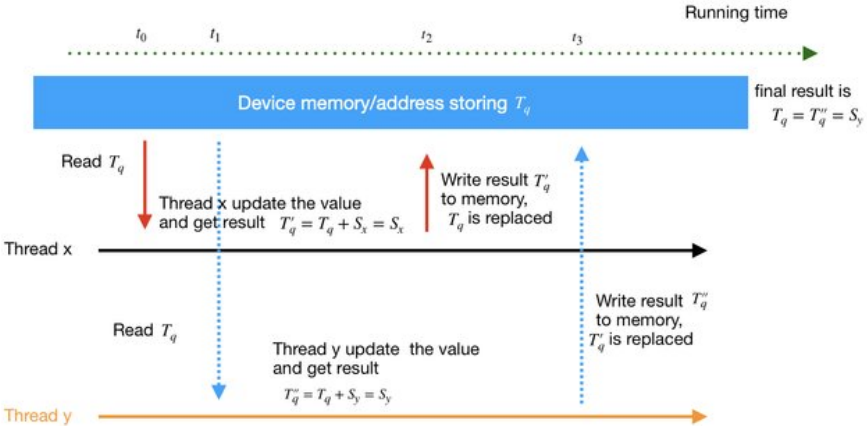
0 Outline

1. Race conditions
2. Exercise

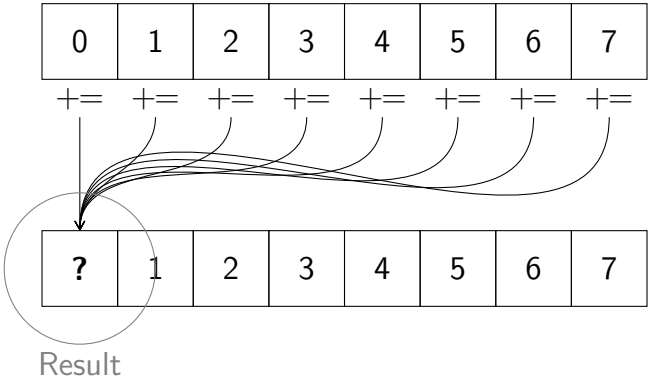
1 Outline

1. Race conditions
2. Exercise

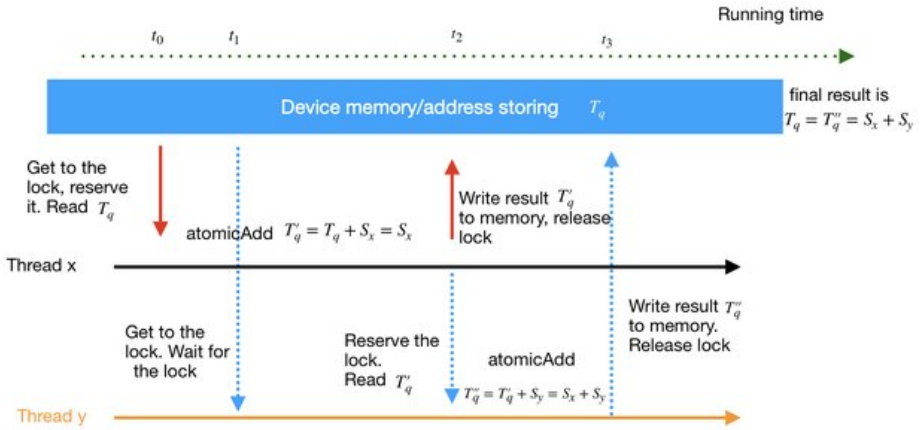
1 Race conditions



1 Addition - an example of possible race conditions



1 Resource locking and waiting as an alternative



1 Atomic operation

An atomic command gets executed without interruption

Threads get **sequential access** to a common resource

1 Atomic operation

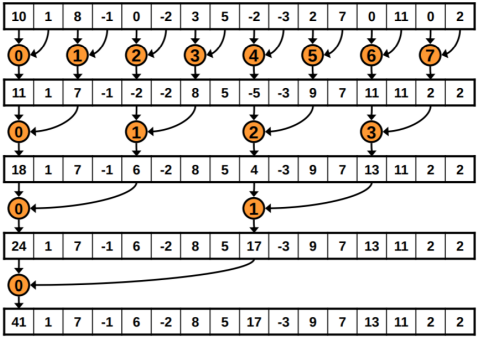
An atomic command gets executed without interruption

Threads get **sequential access** to a common resource

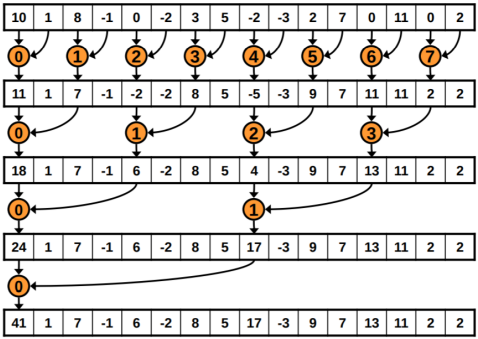
CUDA (and other interfaces) features built-in atomic functions:

- For addition `atomicAdd(...)`
- For subtraction `atomicSub(...)`
- For minimum detection `atomicMin(...)`
- For maximum detection `atomicMax(...)`
- ... full list and reference on p. 187–190 of the programming guide

1 Semi-parallel addition using reduction

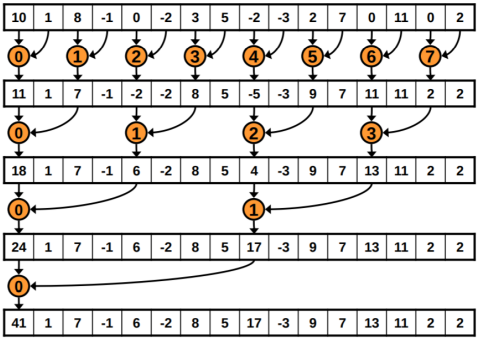


1 Semi-parallel addition using reduction



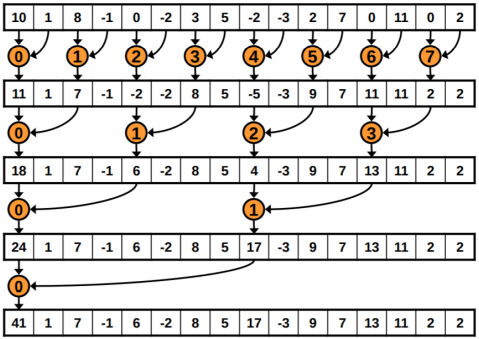
- Ideal for 2^N long inputs

1 Semi-parallel addition using reduction



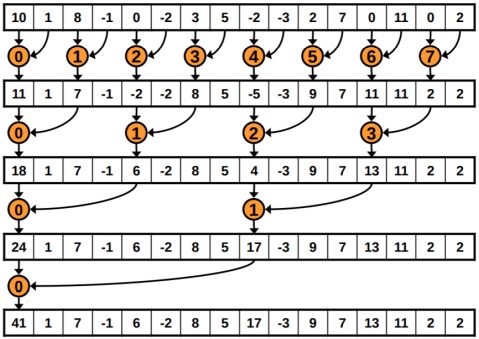
- Ideal for 2^N long inputs
- Complexity $\mathcal{O}(\log_2(N))$

1 Semi-parallel addition using reduction



- Ideal for 2^N long inputs
- Complexity $\mathcal{O}(\log_2(N))$
- Synchronize threads on each reduction step to avoid race conditions (see p. 173)

1 Semi-parallel addition using reduction



- Ideal for 2^N long inputs
- Complexity $\mathcal{O}(\log_2(N))$
- Synchronize threads on each reduction step to avoid race conditions (see p. 173)
- Shifting direction not important

2 Outline

1. Race conditions
2. Exercise

2 Max detection using three approaches

Implement max detection using:

- A nested for loop on the CPU,
- A GPU kernel employing atomic operation, and
- A GPU kernel using reduction.

Use a 1D input array of your liking.

Assess the dependency of the execution time on the input array's length.

Timing examples are provided in the cheat sheet.

2 Tips

Start simple, only consider 1 block!

Many ways to accomplish the same goal

Some ways might use shared memory (more in lab 4)

- Memory is shared per block (all threads in block can access it)
- `extern __shared__ int sdata[];`
- Calling a kernel with shared memory

  `foo<<<nr_blocks, nr_threads, size_shared_memory>>>`

2 Guiding questions

- **Is the result correct?**
- **How does time increase as a function of input size?**
- What happens if the input is larger than 2048 (how would you ensure correct calculation and are there additional synchronization steps)?
- Consider calling the kernel with much less threads than there are data elements. For example, one tenth. How would you ensure all elements get processed?
- How would you tackle a 2D array?